

Resolución: Buscar dos elementos en S que sumen X

Este documento presenta la resolución detallada para determinar si existen dos elementos dentro de un conjunto S de n números reales que sumen exactamente un valor objetivo x , garantizando un coste temporal estricto de $O(n \log n)$.

a) Idea del Algoritmo

Para resolver este problema en $O(n \log n)$, utilizamos una estrategia combinada de **Ordenación** seguida de una búsqueda con **Dos Punteros** (también conocida como búsqueda bidireccional):

1. **Ordenación preliminar:** Primero, ordenamos el conjunto S de menor a mayor. Utilizando un algoritmo de ordenación eficiente (como *MergeSort* o *QuickSort*), este paso consume un tiempo de $O(n \log n)$.
2. **Estrategia de dos punteros:** Una vez ordenado el vector, colocamos dos punteros (índices): uno al principio del vector ($izq = 1$) y otro al final ($der = n$). Sumamos los valores a los que apuntan:
 - Si la suma es igual a x : ¡Hemos terminado! Hemos encontrado la pareja y devolvemos `Verdadero`.
 - Si la suma es mayor que x : Dado que el vector está ordenado, para reducir la suma debemos apuntar a un número más pequeño. Por lo tanto, decrementamos el puntero derecho ($der \leftarrow der - 1$).
 - Si la suma es menor que x : Para incrementar la suma, necesitamos apuntar a un número mayor. Por tanto, incrementamos el puntero izquierdo ($izq \leftarrow izq + 1$).

Este proceso se repite hasta encontrar los elementos o hasta que los dos punteros se crucen, garantizando que exploramos todo el espacio de soluciones de forma eficiente.

b) Algoritmo en Pseudocódigo

A continuación se presenta el pseudocódigo del algoritmo utilizando índices con base 1:

```

Funcion ExisteSumaX(S, n, x)
Inicio
  // Paso 1: Ordenar el conjunto S de menor a mayor
  S_ordenado <- OrdenarMergeSort(S) // Coste: O(n log n)

  // Paso 2: Inicializar punteros en los extremos
  izq <- 1
  der <- n

  // Paso 3: Búsqueda bidireccional
  Mientras izq < der Hacer
    suma_actual <- S_ordenado[izq] + S_ordenado[der]

    Si suma_actual = x Entonces
      Retornar Verdadero // Se encontró la pareja

```

```
Sino Si suma_actual > x Entonces
    der <- der - 1 // Reducir el valor de la suma
Sino
    izq <- izq + 1 // Aumentar el valor de la suma
FinSi
FinMientras

// Si los punteros se cruzan sin éxito, no existen tales elementos
Retornar Falso
FinFuncion
```

d) Análisis de Coste

- **Complejidad Temporal:** $O(n \log n)$.
 - El paso de ordenación toma $O(n \log n)$ en el peor de los casos.
 - El bucle `Mientras` recorre el vector utilizando dos punteros que avanzan el uno hacia el otro, por lo que realiza a lo sumo n iteraciones de coste constante $O(1)$. Esto da un coste de búsqueda de $O(n)$.
 - Por tanto, el coste total es $O(n \log n) + O(n) = O(n \log n)$.
- **Complejidad Espacial:** $O(n)$ si utilizamos un algoritmo de ordenación como *MergeSort* que requiere espacio auxiliar, o bien $O(1)$ adicional (en el sitio) si usamos variantes in-place eficientes.